

◆ What is SAP ABAP?

SAP ABAP (Advanced Business Application Programming) is a programming language used to develop applications in the SAP system.

- It is mainly used for:
 - Custom reports
 - Interfaces
 - Data processing
 - Enhancements in SAP

👉 SAP system is used by companies to manage business processes like Finance, Sales, HR, etc., and **ABAP is the language used to build and customize these applications.**

📌 Key Points:

- ABAP is a **4th Generation Language (4GL)** → Easy to learn
- Works inside SAP system (not standalone like Java/Python) **ABAP programs can only run within the SAP environment**, not independently on your computer like other languages.
- Used for both **backend logic and data handling**

📌 Example:

If business needs:

- Custom invoice report
- Validation before posting
👉 ABAP is used to develop that logic.

📖 What is TCode (Transaction Code) in SAP?

◆ Basic Definition

👉 A **TCode (Transaction Code)** is a **shortcut command** used to quickly open a specific screen or function in SAP.

📁 1 ECC (SAP ECC)

👉 Full form: **ERP Central Component**

✓ **What it is:**

- SAP business system (ERP)

✓ **Database support:**

- Can run on:
 - Oracle
 - SQL Server
 - DB2
 - SAP HANA (later versions)

👉 So:

✓ ECC can use HANA, but it is not HANA itself

2 S/4HANA

👉 Full form: **SAP Business Suite 4 HANA**

✓ **What it is:**

- New generation SAP ERP system

✓ **Important Point:**

👉 Works **ONLY** on **SAP HANA** database

✓ Cannot run on Oracle or others

What is DDIC (Data Dictionary) — Simple Explanation

◆ Basic Definition

DDIC (Data Dictionary) is the **central place in SAP** where all data definitions are stored.

👉 It does NOT store actual business data

👉 It stores **how the data should look and behave**

◆ Simple Real-Life Analogy

Think of DDIC like a **blueprint of a house** 🏠

- Blueprint defines:
 - Number of rooms
 - Size of rooms
 - Structure

👉 But it does NOT contain actual furniture or people

✓ Same way:

- DDIC defines **structure of data**
- Database stores **actual data**

📖 Main Components of DDIC (Data Dictionary)

DDIC has several components that define how data is structured and used in SAP.

◆ 1 Domain

👉 Domain defines the technical properties of a field

It includes:

- Data type (CHAR, NUMC, DEC, etc.)
- Length (e.g., 10, 15)
- Decimal places
- Value range (optional)

✓ Example:

Amount → DEC (13,2)

👉 Think:
“How the data is stored technically”

The screenshot shows the 'Dictionary: Display Domain' dialog box. The 'Domain' field contains 'EBELN' and is marked as 'Active'. The 'Short Description' is 'Purchasing document number'. The 'Format' tab is selected, showing 'Data Type' as 'CHAR' (Character String), 'No. Characters' as '10', and 'Decimal Places' as '0'. The 'Output Characteristics' section shows 'Output Length' as '10', 'Routine' as 'ALPHA', and an unchecked 'Sign' checkbox.

◆ ② Data Element

Data Element includes:

- Field labels (Short, Medium, Long)
- Description
- Its Domain + Short Desc

The screenshot shows the 'Dictionary: Display Data Element' dialog box. The 'Data element' field contains 'EBELN' and is marked as 'Active'. The 'Short Description' is 'Purchasing Document Number'. The 'Data Type' tab is selected. Under 'Elementary Type', the 'Domain' radio button is selected, with 'EBELN' in the domain field and 'Purchasing document number' as the description. The 'Data Type' is 'CHAR' (Character String) and the 'Length' is '10'. Other options like 'Predefined Type', 'Reference Type', and 'Referenced Type' are not selected.

③ Table

Table stores actual business data

Types:

SAP mainly has 3 types (but today you mostly use one):

- Transparent Table  (Most important)
- Pooled Table  (Old)
- Cluster Table  (Old)

 Definition:

A transparent table has a **1:1 relationship with a database table**

 What it means:

- One SAP table = One database table
 - Structure is same in SAP and DB
-

 Example:

- EKKO → Purchase Order Header
 - EKPO → Purchase Order Item
-

 How it works:

- You define table in DDIC
 - SAP creates same table in database
 - Data is stored directly
-

 Key Features:

- Fast access 
- Easy to understand
- Supports SQL queries directly
- Used in all modern SAP systems

♦  Pooled Table

👉 **Definition:**

Multiple small tables are stored in **one single database table (pool)**

✓ **What it means:**

- Many SAP tables → One DB table
-

✓ **Why it was used:**

- To save space
 - For small tables with less data
-

✗ **Problems:**

- Complex structure
- Slower performance
- Cannot directly use SQL easily

③ Cluster Table

👉 **Definition:**

Multiple related tables are stored together in **one database table (cluster)**

✓ **What it means:**

- Related tables grouped together
 - Stored in compressed form
-

✓ **Example:**

- BSEG (Accounting data – classic example)
-

✓ **Why it was used:**

- Reduce storage
 - Improve read for related data
-

✗ Problems:

- Difficult to access directly
- Not compatible with modern databases (like SAP HANA)

In modern SAP (especially with SAP HANA):

- ✗ Pool & Cluster tables are **deprecated**
 - ✔ Converted into **Transparent tables**
-

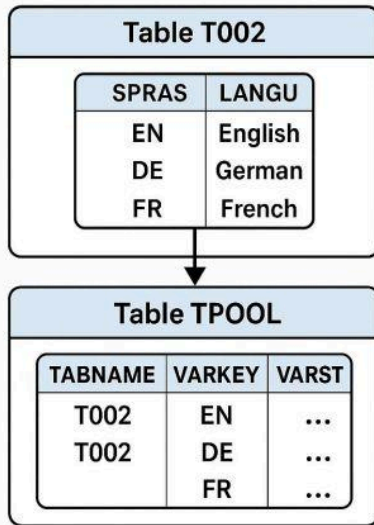
◆ Quick Comparison

Feature	Transparent Table	Pooled Table	Cluster Table
Storage	1:1	Many → One	Many → One
Performance	Fast	Slow	Medium
Complexity	Simple	Complex	Complex
Usage Today	✔ Yes	✗ No	✗ No

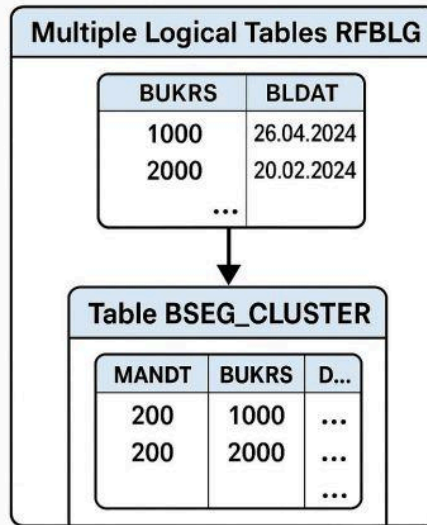
◆ Basic Idea

- **Pooled Table** → Many **small, unrelated tables** stored together
- **Cluster Table** → Many **related tables** stored together

Pooled Table



Cluster Table



 **If pooled tables are unrelated, why store them together?**

◆ Short Answer

👉 To save database space and reduce overhead (in older systems)

◆ The Real Reason

In older SAP systems, databases were:

- Expensive 💰
- Limited in storage
- Slower in handling many small tables

! Problem

There were **hundreds of very small tables** like:

- Config tables
- System tables

- Control tables

👉 Each table:

- Needs storage
 - Needs metadata
 - Needs database resources
-

♦ Solution → Pooling

👉 SAP idea was:

“Why create 100 small tables separately...
when we can store them inside **one big table**?”

✓ So what they did:

- Created **one physical table (pool)**
- Stored multiple small tables inside it

👉 Example:

- TSTC
- Other small tables

➡ All stored in one pool table

♦ Simple Analogy

👉 Without pooling:

- 100 small boxes 📦📦📦 (wastes space)

👉 With pooling:

- 1 big box containing all small items 📦

- ✓ Saves space
- ✓ Less database objects

Transparent	Pool	Cluster
Contain a single table. Used to store master data	They are used to hold a large number of very small tables(stores customizing data or system data)	They are used to hold data from a few number of large tables.(stores system data)
It has a one-to-one relationship with a table in the database	It has a many-to-one relationship with a table in the database	It has a many-to-one relationship with table in the database
For each transparent table there is one associated table in the database	It is stored with other pooled tables in a single table called table pool in the database	Many cluster tables are stored in a single table in the database called a table cluster
The database table has the same name, same number of fields and the fields have the same names	The database table has different name, different number of fields and fields have different names	The database table has different name, different number of fields and fields have different names
There is only a single table	Table pools contain more tables than table clusters	Contains less tables than table pools
Single table can have one or more primary key	Primary key of each table does not begin with same fields or fields	Primary key of each table begins with same fields or fields
Secondary indexes can be created	Secondary indexes cannot be created	Secondary indexes cannot be created
They can be accessed using open and native SQL	They can be accessed using open SQL only	They can be accessed using open SQL only
USE: They are used to hold master data e.g. Table vendors or table of customers. Example of transaction data is orders placed by customers	USE: They reduce the amount of database resources needed when many small tables have to be opened at the same time	USE: They would be used when the tables have primary key in common and data in these tables are all accesses simultaneously

◆ What is a Transparent Table?

👉 A **Transparent Table** is a table in SAP that has a **direct 1:1 relationship** with a database table

✓ Meaning:

- One SAP table = One database table
- Same structure in both SAP and DB

◆ Key Features

✓ 1. Direct Mapping

- SAP table = Database table

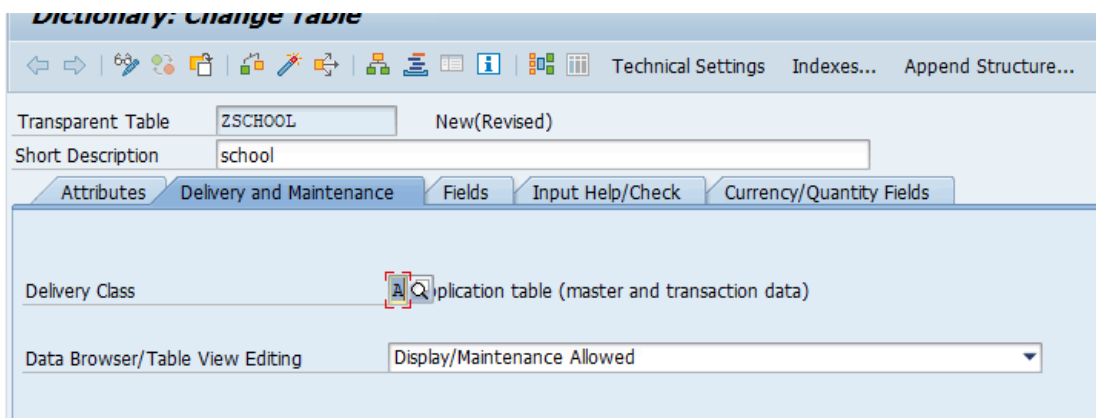
✓ 2. High Performance

- Fast read/write

✓ 3. Easy to Use

- Simple structure
- Easy debugging

✓ 4. Supports SQL



Delivery Class in SAP (Full Explanation)

👉 Delivery Class defines:

- Type of data stored
- Who maintains it
- How it behaves during transport (DEV → QA → PROD)

◆ Delivery Classes

A – Application Table

👉 Stores business data (real data) , Application tables store real business data, which is different in each system (DEV, QA, PROD)

✓ Used for:

- Master data (Customer, Vendor)
- Transaction data (PO, Invoice)

✓ Example:

- EKKO (PO Header)
- KNA1 (Customer)

👉 Data is:

- Created by users
- NOT transported

1 Master Data

👉 Data used frequently but changes rarely.

Examples:

- Employee Data
- Vendor Data
- Material Master

✓ Stable data

2 Transaction Data

👉 Daily changing business records.

Examples:

- Banking transaction
- Revenue transaction
- PO creation
- Invoice posting

✓ Continuously increasing

◆ So What Does Delivery Class A Mean?

👉 It means your table can store:

- ✓ Master Data
- ✓ Transaction Data

Both are called:

👉 **Application Data**

Data Browser / Table View Editing

👉 This controls whether you can **view or edit table data directly using SE16 / SM30**

◆ In your case:

👉 **Display/Maintenance Allowed**

✓ What it means:

- You can:
 - View data  (SE16)
 - Edit data  (SM30 or SE16N with edit)

2. Display/Maintenance Not Allowed

 User cannot:

- View data in SE11/SE16
- Generate maintenance screens
- Maintain data in SM30

Simple Meaning:

Table is fully protected.

Example:

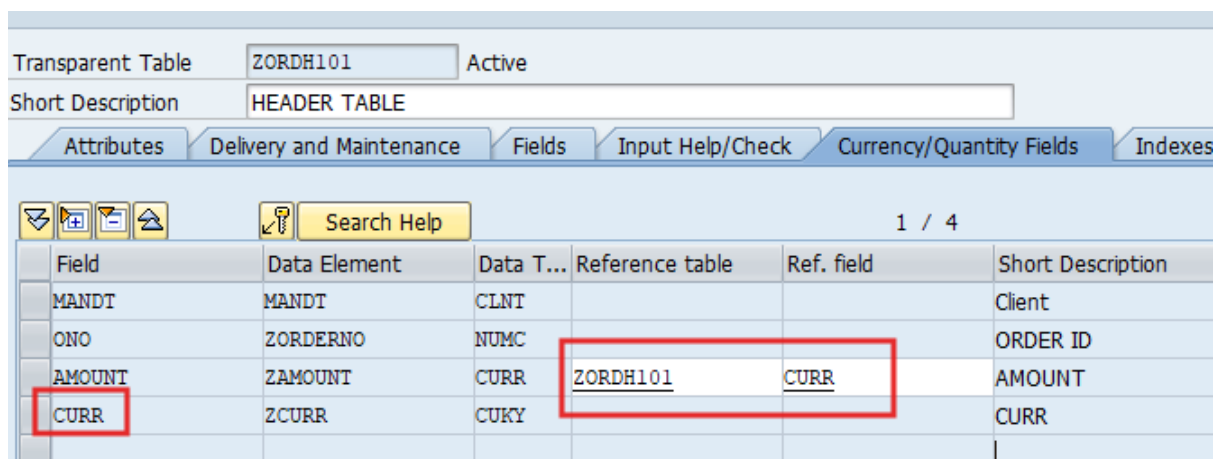
Sensitive system tables.

3. Display/Maintenance Allowed With Restrictions

✓ User can:

- View data in SE11 and SE16

👉 Direct maintenance using SM30 is not allowed.



The screenshot shows the SAP configuration for table ZORDH101. The table is active and has a short description of 'HEADER TABLE'. The 'Fields' tab is selected, showing a list of fields. The 'Curr' field is highlighted with a red box, and its reference table 'ZORDH101' and reference field 'CURR' are also highlighted with a red box.

Field	Data Element	Data T...	Reference table	Ref. field	Short Description
MANDT	MANDT	CLNT			Client
ONO	ZORDERNO	NUMC			ORDER ID
AMOUNT	ZAMOUNT	CURR	ZORDH101	CURR	AMOUNT
CURR	ZCURR	CUKY			CURR

Why Table Activated Without Reference Field/Table

Simple Explanation

SAP asks for **Reference Table** and **Reference Field** only for special fields like:

- Currency Amount (**CURR**)
- Quantity (**QUAN**)

In our table:

Field **Data Type**

AMOUNT DEC

CURR CUKY

Here, **AMOUNT** is **DEC** type.

DEC means a normal decimal number.

Example:

5000.25

Since it is just a normal number, SAP does not need any reference field.

When Reference Field is Required

If **AMOUNT** datatype is **CURR**, then SAP needs to know:

Which currency belongs to this amount?

Example:

500 INR

500 USD

Then we must connect:

Amount Field Currency Field

AMOUNT CURR

Foreign Key Relationship in SAP

A Foreign Key is used to connect two tables in SAP.

It ensures that data entered in one table already exists in another table.

Criteria for Foreign Key

To create a foreign key successfully:

1. Check Table must contain:

- Same field
- Same datatype/domain
- Field should be a KEY field

2. Foreign Key Table field must:

- Match the check table field

Primary Key

A primary key uniquely identifies each record in a table.

Foreign Key

A foreign key is used to create relationship between two tables. The value entered in the child table must already exist in the parent table.

Difference Between Primary Key and Foreign Key

Primary Key	Foreign Key
Uniquely identifies each record in a table	Creates relationship between two tables
Cannot contain duplicate values	Duplicate values allowed
Cannot be NULL	Can be NULL
Exists in parent table	Exists in child table
Used to uniquely identify data	Used to validate/reference data

A Foreign Key creates a relationship between:

- **Parent Table** (Main table)
- **Child Table** (Dependent table)

What is Client in SAP?

◆ Basic Definition

👉 A **Client** in SAP is like a **separate environment/business unit** inside one SAP system.

Each client has:

- Its own data
 - Its own users
 - Its own settings
-

◆ Real-Life Analogy

Think of SAP system like a **big apartment building** 🏢

Inside the building:

- Flat 800
- Flat 810
- Flat 900

👉 Each flat is separate.

- ✓ Same building
- ✓ Different people/data

What is MANDT?

👉 **MANDT** means:

Mandant = Client

It is a field used to identify **which client the data belongs to**.

◆ Example Table

Suppose table:

ZSTUDENT

Contains:

◆ What Happens?

If user logs into Client 800

👉 They see only:

Ram

If user logs into Client 900

👉 They see only:

Shyam

◆ Why SAP Uses Client?

👉 To separate data safely.

Different clients can be used for:

- Development
- Testing
- Production
- Different companies

without mixing data.



Client Dependent Table

👉 If table contains MANDT field, then table is called Client Dependent Table

✓ Data changes based on client login.



Client Independent Table

👉 If table does NOT contain MANDT then table is Client Independent Table

✓ Same data visible in all clients.

◆ Example

✓ Client Dependent

Business tables:

- Customer data
- Purchase Orders

Need separate data per client.

✓ Client Independent

System/config tables:

- Country codes
- Language settings

Table Maintenance Generator (TMG) in SAP ABAP

◆ What is Table Maintenance Generator (TMG)?

👉 **Table Maintenance Generator (TMG)** is a SAP tool used to create **screens for maintaining table data** without writing ABAP code.

✓ It helps users:

- Insert data
- Update data
- Delete data
- Display data

directly through SAP screens.

◆ Why TMG is Used?

Normally, if we create a table:

- Data exists in DB
- But no screen is available to maintain it

👉 TMG automatically creates:

- Maintenance screens
- Function modules
- Standard logic

So users can maintain data easily.

◆ Types of Maintenance

1 One Step

👉 All data maintained in one screen

✓ Simple & most common

2 Two Step

👉 First screen:

- List overview

👉 Second screen:

- Detail view

✓ Used for complex tables

Important Points Related to TMG

✓ 1. Table Maintenance Option

👉 In table settings:

Data Browser/Table View Maintenance

must be:

Display/Maintenance Allowed

Otherwise:

✗ SM30 maintenance will not work.

✓ 2. Adding New Fields After TMG Generation

👉 If new fields are added later to the table:

✗ They will NOT appear automatically in SM30.

✓ Solution:

- Delete old TMG
 - Regenerate TMG again
-

✓ 3. Deleting TMG

👉 Deleting TMG:

✗ Does NOT delete table data.

Only generated maintenance objects are deleted.

✓ 4. To Delete TMG

👉 Open table in:

Change Mode

Otherwise:

✗ Delete button will not be visible.



Structures in SAP DDIC

◆ What is Structure?

👉 Structure is a collection of fields/columns of:

- Different data types
OR
- Similar data types

✓ It groups related fields together logically.

✗ Structure does NOT store data physically in database.

◆ Example of Structure

Structure : ZEMP

Field	Data Type
EID	N(10)
ENAME	C(40)

◆ Difference Between Table & Structure

Table

Structure

Table stores data	Structure does not store data
Physical DB table created	No DB table created
Used for permanent storage	Used for temporary data
Example: EKKO, MARA	Example: BAPI structures



Include Structure

◆ What is Include Structure?

👉 Include is a reusable structure.

✓ We can insert/include the same structure into multiple tables or structures.

✓ Key Points:

- ✓ It is a **reusable structure**
- ✓ Can be inserted **anywhere** in the table
- ✓ Mostly used in **custom (Z) tables**
- ✓ Fields behave as if they are part of the table

◆ Example

Include Structure : ZEMP_COMMON

Field

CREATED_BY

CREATED_ON

👉 This include can be used in:

- ZEMPLOYEE
 - ZSTUDENT
 - ZVENDOR
-

Append Structure

♦ What is Append Structure?

👉 Append Structure is used to add custom fields to an existing table or structure.

✅ Key Points:

1. ❌ It is **NOT reusable like INCLUDE**
 2. ✅ Fields are always added **at the END**
 3. ✅ Used for both:
 - Custom tables
 - Standard SAP tables (very important use case)
 4. ✅ Used when you **cannot modify original table**
 - APPEND is used to **extend a table without modifying it directly**
-

♦ Example

Suppose SAP standard table:

MARA

We add custom field:

ZZCOUNTRY

using Append Structure.

👉 New field added at the end of MARA table.

♦ Difference Between Include & Append

Include Structure	Append Structure
Reusable	Not reusable
Can insert anywhere	Always added at end
Mostly for custom tables	Used for SAP & custom tables
Used for common fields	Used for enhancements

◆ 5. SE14 (Database Utility) ★

👉 SE14 is used to adjust or repair the database table after structure changes

🧠 When Do We Use SE14?

Use SE14 when you get errors like:

- ✗ *"Structure change at field level"*
- ✗ *"Convert table required"*
- ✗ Activation errors after changing:
 - Fields
 - Key fields
 - Include
 - Append

